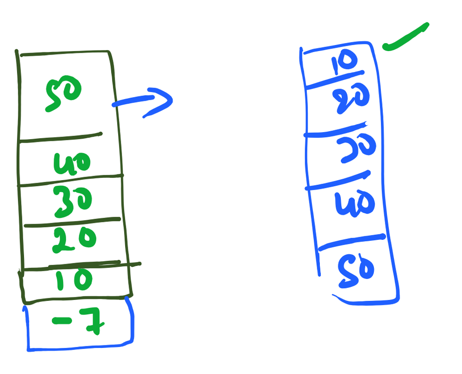
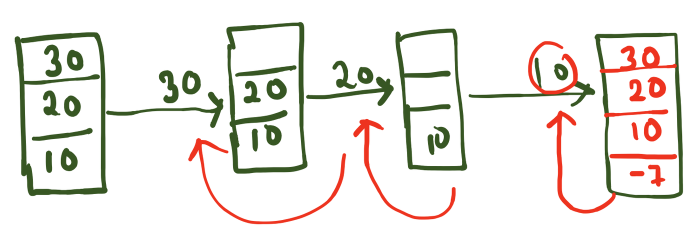
Lec-25

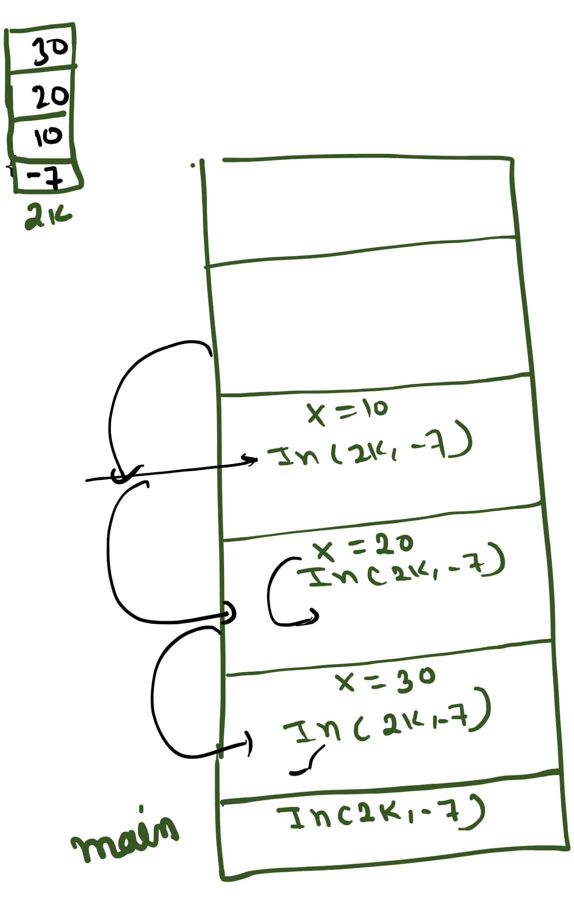
24/05/26 9:47 AM

```java
import java.util.Stack;

public class Insert_Buttom {

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        Stack<Integer> st = new Stack<>();
        st.push(10);
        st.push(20);
        st.push(30);
        st.push(40);
        st.push(50);
        System.out.println(st);
        Insert(st, -7);
        System.out.println(st);

    }

    public static void Insert(Stack<Integer> st, int item) {

    }

}
```

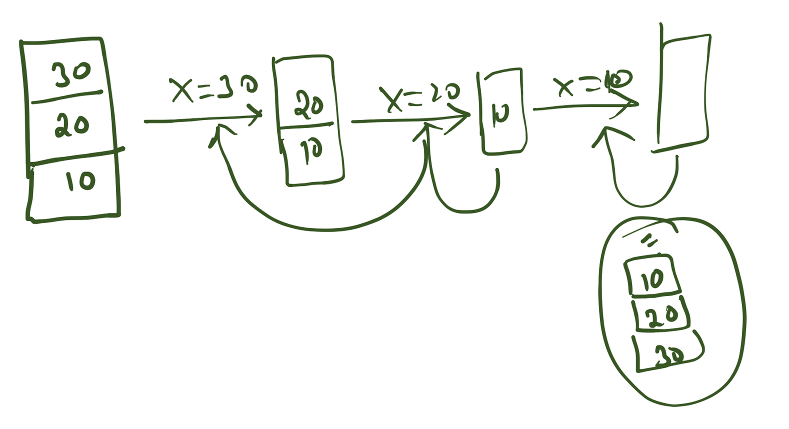




```java
public static void main(String[] args) {
        // TODO Auto-generated method stub
        Stack<Integer> st = new Stack<>();
        st.push(10);
        st.push(20);
        st.push(30);
        st.push(40);
        st.push(50);
        System.out.println(st);
        Insert(st, -7);
        System.out.println(st);

    }

    public static void Insert(Stack<Integer> st, int item) {
        if (st.isEmpty()) {
            st.push(item);
            return;
        }
        int x = st.pop();
        Insert(st, item);
        st.push(x);
    }
```



```java
private static void Reverse(Stack<Integer> st) {
        // TODO Auto-generated method stub
        if (st.isEmpty()) {
            return;
        }
        int x = st.pop();
        Reverse(st);
        st.push(x);

    }
```



2375. Construct Smallest Number From DI String

Solved

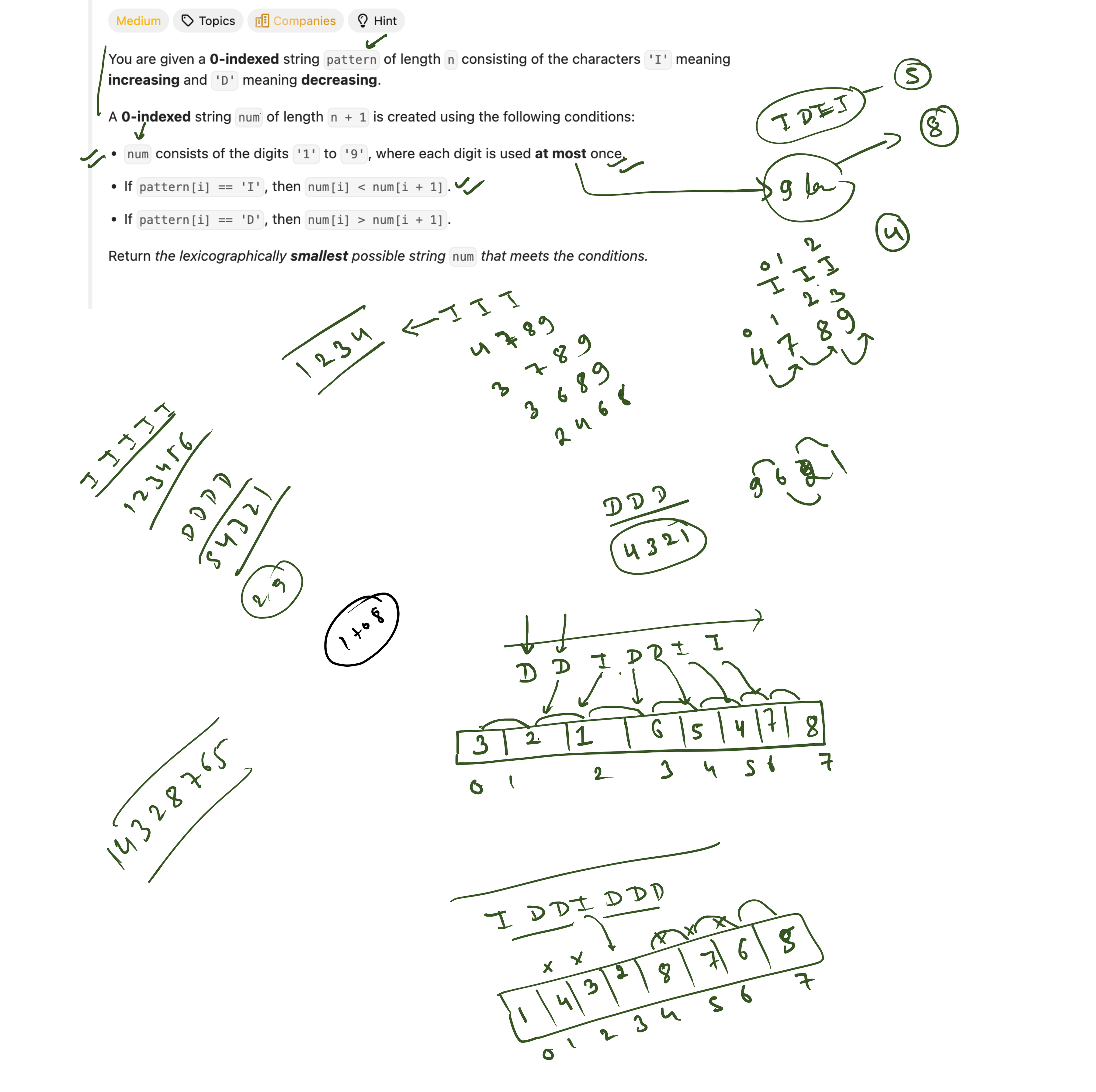
Medium · Topics · Companies · Hint

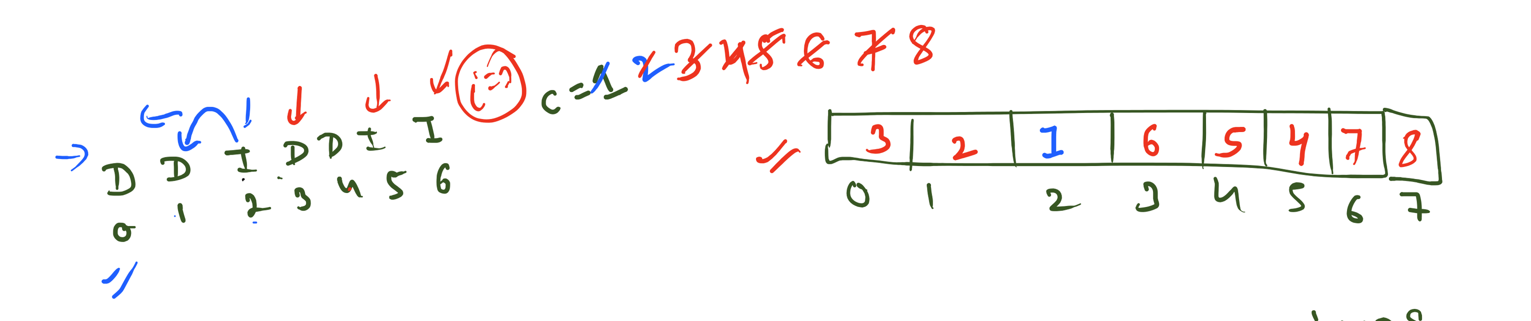
You are given a **0-indexed** string `pattern` of length `n` consisting of the characters `'I'` meaning **increasing** and `'D'` meaning **decreasing**.

A **0-indexed** string `num` of length `n + 1` is created using the following conditions:

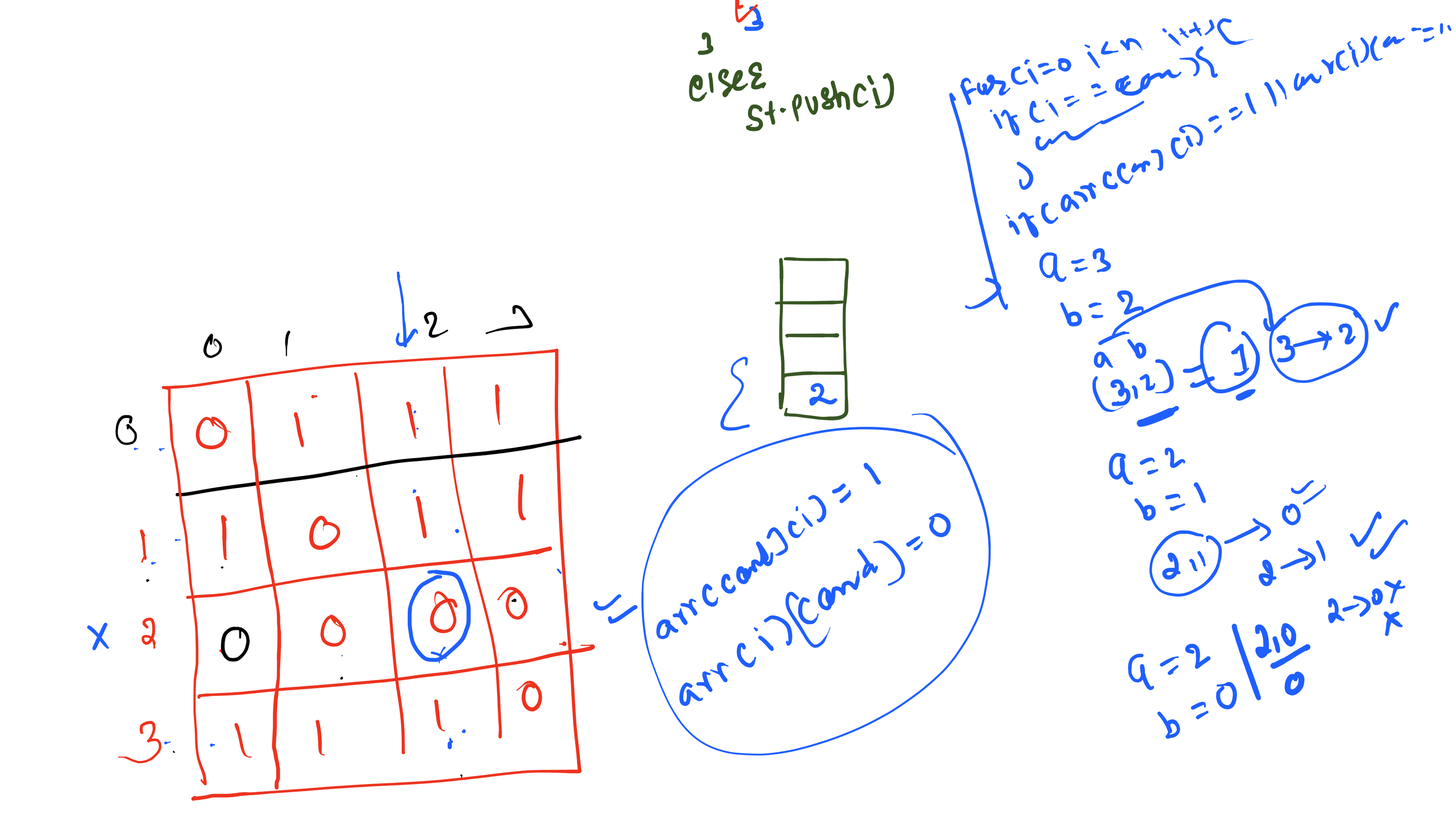
- `num` consists of the digits `'1'` to `'9'`, where each digit is used **at most** once.
- If `pattern[i] == 'I'`, then `num[i] < num[i + 1]`.
- If `pattern[i] == 'D'`, then `num[i] > num[i + 1]`.

Return *the lexicographically* ***smallest*** *possible string* `num` *that meets the conditions.*



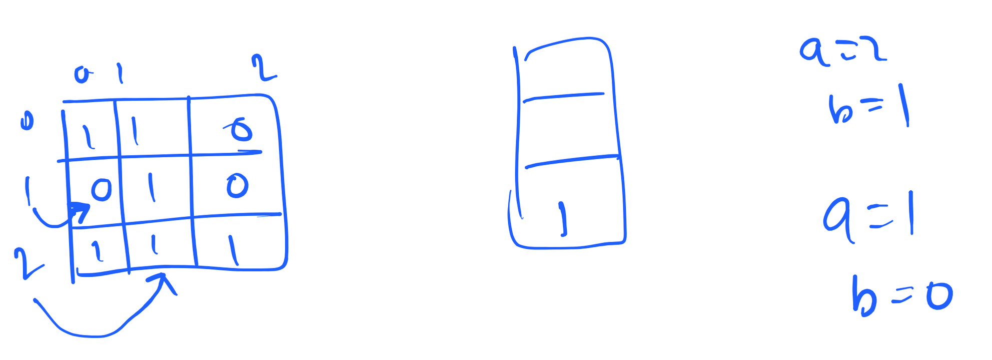


```
for (i=0  i<=s.length()  i++){
if (i==s.len() || s.charAt(i) == 'I'){
    ans[i]=c++;
    while (!st.isEmpty()){
      ans[st.pop()] = c++
    }
}
else{
    st.push(i)
```



arr[cand][i] = 1

arr[i][cand] = 0



Given a string `s` containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.